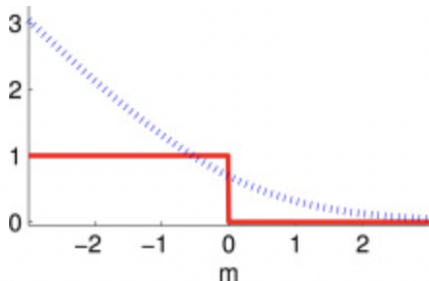


Remarks on KNN

Procheta Sen



Complexity of k -NN

- ▶ **Training:** store entire training set
- ▶ **Classification:** for an input object $\bar{X}'$,
 - ▶ find in the training set the k closest (nearest) objects to $\bar{X}'$
 - ▶ find the majority label among those nearest neighbours
 - ▶ the majority label is predicted to the test instances $\bar{X}'$

Training dataset

n objects and d features

Time: store data (constant per object)

Space: $n \times d$

Test time

For each test example we need to find k nearest neighbours. The test example must be compared with every training example.

Complexity of k -NN (Continued)

- ▶ **Training** is computationally cheap
- ▶ **Classification** is computationally expensive

In practice the **classification** time is much more important than the **training** time

Possible solution: use Approximate Nearest Neighbor (ANN) algorithms that can accelerate the nearest neighbor lookup in a dataset at the expense of query accuracy (e.g., FLANN — Fast Library for Approximate Nearest Neighbors).

Inductive Bias and Feature Importance

k -NN classifier assumes

- ▶ that nearby points should have the same label
- ▶ all features are equally important!
(if there are only a few relevant and many irrelevant features in a dataset, the k -NN classifier is likely to perform poorly)

Summary on k -NN classifier

- ▶ Preprocess your data: normalise the features in your dataset to have zero mean and unit variance.
- ▶ If your data is very high-dimensional, consider using dimensionality reduction.
- ▶ Split your training data into training (50–90%) and validation (10–50%).
- ▶ If the validation dataset is too small, split your training data into folds and perform cross-validation.
- ▶ Train and evaluate the k -NN classifier on the validation data for many choices of k and for different types of distances (start with L^1 and L^2).